



University of Massachusetts Amherst

Department of Microbiology¹

Department of Electrical and Computer Engineering²

3D Skeletonization

Notebook Documentation

Version 1.0 · May 2026

Prepared by: Cooper Richman²

Supervisors

Dr. Sang Hyun Lee¹

Dr. Bill Leonard²

Contents

1	Introduction	1
1.1	Key Capabilities	1
1.2	Dependencies	2
2	Notebook Cells Overview	3
2.1	Cells 1–2: 3D Voxelization from <code>.vtk</code>	3
2.2	Cell 3: Hybrid 2D-to-3D Skeleton with Gap Filling	3
2.3	Cell 4: Full 3D Skeletonization Pipeline (Lee Algorithm)	4
2.4	Cell 5: Voxelization to <code>.tiff</code> and Skeletonization Save	4
2.5	Cells 6–7: Smoothing Experiments	5
3	Typical Workflow	6
4	Limitations & Notes	7
5	Change Log	9

Chapter 1

Introduction

The **3D Skeletonization** notebook (`3D Skeletonization.ipynb`) provides a pipeline for computing true volumetric skeletal representations of mycelial fungal networks. Unlike the 2D skeletonization approach, which flattens three-dimensional data via a max projection, the cells in this notebook operate directly on the full 3D voxel grid. This preserves spatial structure in all three axes and opens the possibility of measuring branch angles and growth speeds in three dimensions.

Input data may be cleaned `.vtk` mesh files or pre-computed voxelization `.tiff` stacks. The notebook also explores smoothing strategies that improve skeleton continuity before and after the skeletonization step.

1.1 Key Capabilities

- Interactive 3D voxelization of `.vtk` meshes (with and without an intermediate `.stl` file).
- True 3D skeletonization using the Lee algorithm (`skimage.morphology.skeletonize(..., method="lee")`).
- Hybrid 3D skeleton: 2D top-down skeleton mapped back to 3D using the geometric center of each pixel's Z column.
- Gap interpolation along X, Y, and Z axes to reduce discontinuities in the mapped 3D skeleton.
- Gaussian smoothing of the voxel volume at multiple σ values prior to skeletonization, with simultaneous Napari visualization of all smoothed results.

- Post-skeletonization smoothing: Gaussian blur applied to a 1-voxel-thick skeleton followed by re-skeletonization to reduce jagged artifacts.
- Saving the skeletonized volume as a `.tiff` file for use in downstream tools.

1.2 Dependencies

Package	Install command	Role
<code>numpy</code>	<code>pip install numpy</code>	Array operations and volume manipulation
<code>pyvista</code>	<code>pip install pyvista</code>	Loading <code>.vtk</code> meshes
<code>trimesh</code>	<code>pip install trimesh</code>	Voxelization and optional STL export
<code>scikit-image</code>	<code>pip install scikit-image</code>	3D Lee skeletonization
<code>scipy</code>	<code>pip install scipy</code>	Gaussian smoothing via <code>ndimage.gaussian_filter</code>
<code>napari</code>	<code>pip install napari</code>	3D interactive volume visualization
<code>tifffile</code>	<code>pip install tifffile</code>	Reading and writing <code>.tiff</code> volume stacks

Chapter 2

Notebook Cells Overview

Each cell is self-contained and can be run independently. They are grouped below by the approach they implement.

2.1 Cells 1–2: 3D Voxelization from `.vtk`

Two equivalent approaches for converting a cleaned `.vtk` mesh into a 3D voxel occupancy grid:

- **Cell 1 (with STL intermediate):** Saves the mesh as a temporary `temp_mesh.stl` file and loads it with Trimesh for voxelization. The result is displayed as a voxel box mesh in a Trimesh viewer.
- **Cell 2 (in-memory):** Extracts vertices and faces directly from the PyVista mesh object and constructs a Trimesh object without writing to disk. This is the preferred approach as it avoids file I/O.

Both cells expose the `pitch` parameter (default: `1`), which controls voxel size. The result of voxelization in either cell can be directly passed to the skeletonization cells below.

2.2 Cell 3: Hybrid 2D-to-3D Skeleton with Gap Filling

Computes a 2D top-down skeleton and maps it back into 3D space by placing each skeleton pixel at the geometric center of its occupied Z column. Small gaps (up

to 2 voxels) are then interpolated along all three axes independently. The result, along with the raw (pre-gap-fill) skeleton and the original voxelization, is visualized in Napari.

Napari 3D mode: When Napari opens, it defaults to 2D view. Click the square/cube toggle button in the bottom-left corner of the Napari window to switch to 3D rendering mode.

2.3 Cell 4: Full 3D Skeletonization Pipeline (Lee Algorithm)

The most complete single-cell pipeline. Loads a `.vtk` file, converts it to a Trimesh object in memory, voxelizes it, fills the interior so the volume is topologically solid, and applies the Lee 3D thinning algorithm. Both the voxelized volume and the resulting skeleton are displayed side by side in a Napari 3D viewer.

Parameter	Default	Description
<code>vtk_path</code>	(user-defined)	Path to the cleaned <code>.vtk</code> file.
<code>pitch</code>	1.0	Voxel size in mesh units.
<code>process</code>	True	When True, Trimesh performs mesh cleanup (triangulation fixup, duplicate removal) before voxelizing.

2.4 Cell 5: Voxelization to `.tiff` and Skeletonization Save

Loads a `.vtk`, voxelizes it, runs the Lee algorithm, and saves the resulting skeleton volume as a `.tiff` stack. The output file path is user-configurable. After saving, the skeleton is displayed as an interactive Trimesh voxel viewer.

Output file: The saved `.tiff` encodes the binary skeleton as an 8-bit image (skeleton voxels = 255, background = 0). This file can be loaded by subsequent smoothing cells.

2.5 Cells 6–7: Smoothing Experiments

Two smoothing strategies are explored:

- **Cell 6 — Pre-skeletonization smoothing:** Applies Gaussian smoothing to the original voxel volume at multiple σ values ($\sigma \in \{1, 2, 3, 4, 5, 6\}$), thresholds back to binary, and skeletonizes each result. All smoothed skeletons are loaded into Napari simultaneously for direct comparison. This approach can reduce branching artifacts from mesh noise.
- **Cell 7 — Post-skeletonization smoothing:** Applies a Gaussian blur (configurable σ , default 1.5) to an already 1-voxel-thick skeleton, thresholds the float field, and re-skeletonizes. Both the original and smoothed skeletons are displayed as point layers in Napari for direct comparison.

Chapter 3

Typical Workflow

- (i) **Voxelize.** Run Cell 2 (in-memory approach) on a cleaned `.vtk` file to confirm the voxelization looks correct in the Trimesh viewer.
- (ii) **Skeletonize.** Run Cell 4 to apply the Lee 3D algorithm. Inspect both the volume and the skeleton in Napari (ensure 3D mode is enabled).
- (iii) **Save.** Run Cell 5 to write the skeleton volume to a `.tiff` file for archiving or downstream use.
- (iv) **Smooth if needed.** If the skeleton shows discontinuities or excessive branching, run Cell 6 to compare pre-smoothed skeletons across σ values, or Cell 7 to smooth an existing skeleton. Choose the σ that best preserves true branch topology while removing noise.

Chapter 4

Limitations & Notes

3D Skeletonization Inherent Issues

3D skeletonization provides a promising avenue to conduct fungal mycelial analysis in three dimensions, but doing so can be quite difficult. Even with a correct and complete 3D skeletonization, there are often spurious branches that are mathematical artifacts that cannot easily be distinguished from their real counterparts. Solutions such as pruning may help to mediate this, but it is hard to know what is real and what is an artifact of the skeletonization algorithm.

3D Skeletonization in FIJI

3D skeletonization can also be achieved using FIJI. From the testing that has been done it seems to give the exact same result as doing it through Python, but the option is there. The input for FIJI is a filled in 3D voxelization, and to skeletonize it, navigate to Plugins > Skeleton > Skeletonize (2D/3D). Then to view the skeletonization (it can sometimes be hard to see, as it is one pixel wide) navigate to Plugins > 3D Viewer.

Lee Algorithm Computation Time

The Lee 3D thinning algorithm is computationally intensive. Large volumes (e.g., $500 \times 500 \times 200$ voxels) may take several minutes to process. The terminal will print a progress message when skeletonization begins. Do not interrupt the kernel during this step.

Hybrid 3D Skeleton Gaps

The hybrid approach in Cell 3 maps a 2D skeleton to 3D using the geometric center of each Z column. Hyphae that cross one another in the Z direction will share the same (x,y) column, meaning only one of the two branches can be correctly placed. Gap filling partially mitigates this but cannot fully resolve topological crossings in 3D.

Interior Filling Required for Lee Algorithm

The Lee algorithm requires a solid (filled) binary volume. If the voxelized mesh has only a thin surface shell (common for surface mesh inputs), the skeleton will be placed on the shell rather than the volumetric center. Cell 4 calls `voxels.fill()` explicitly to address this.

Smoothing trade-offs: Increasing σ in Cells 6 or 7 merges fine branches and reduces topological complexity. This can be desirable for noisy data but may inadvertently remove real biological branching. Always compare smoothed results against the unsmoothed skeleton.

Chapter 5

Change Log

- **February 25, 2026:** File created from parts of earlier notebooks, consolidated in one place. (C. Richman)
- **May 20, 2026:** Continued tidying for final repository. (C. Richman)